

EC2x&EG9x&EG2x-G&EM05 系列

QuecOpen 语音服务 API 参考手册

LTE Standard 模块系列

版本：1.0

日期：2022-01-16

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-12-07	Colin CUI	文档创建
1.0	2022-01-16	Colin CUI	受控版本

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
1.1. 适用模块	7
2 语音服务相关 API.....	8
2.1. 头文件.....	8
2.2. 函数概览.....	8
2.3. 函数描述	9
2.3.1. QL_Voice_Call_Client_Init	9
2.3.1.1. voice_client_handle_type.....	9
2.3.2. QL_Voice_Call_Client_Deinit.....	9
2.3.3. QL_Voice_Call_AddStateHandler.....	10
2.3.3.1. QL_VoiceCall_StateHandlerFunc_t.....	10
2.3.3.2. ql_mcm_voice_call_state_t.....	11
2.3.4. QL_Voice_Call_RemoveStateHandler.....	12
2.3.5. QL_Voice_Call_Start.....	12
2.3.6. QL_Voice_Call_End	13
2.3.7. QL_Voice_Call_Answer	13
2.3.8. QL_Voice_Call_SetWaiting.....	14
2.3.8.1. E_QL_MCM_VOICE_CALL_WAITING_SERVICE_T	14
2.3.9. QL_Voice_Call_GetWaitingStatus	15
2.3.10. QL_Voice_Call_Hold.....	15
2.3.11. QL_Voice_Call_UnHold	16
2.3.12. QL_Voice_Call_CancelDial.....	16
2.3.13. QL_Voice_Call_Dtmf.....	16
2.3.14. QL_Voice_Call_SetForwarding.....	17
2.3.14.1. E_QL_MCM_VOICE_CALL_SERVICE_T.....	18
2.3.14.2. E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T	18
2.3.15. QL_Voice_Call_GetForwardingStatus.....	19
2.3.15.1. ql_mcm_voice_call_forwarding_status_list_t.....	19
2.3.15.2. QL_MCM_VOICE_CALL_FORWARDING_STATUS_T	20
2.3.15.3. QL_MCM_VOICE_CALL_FORWARDING_INFO_T	20
2.3.15.4. QL_MCM_VOICE_CALL_FORWARDING_TYPE_T.....	21
3 语音服务使用步骤示例.....	22
4 编译说明.....	23
5 语音服务演示步骤.....	24
5.1. 拨打电话	24
5.2. 接听并挂断电话	24

5.3. 保持呼叫	25
6 附录 参考文档及术语缩写	27

表格索引

表 1: 适用模块	7
表 2: 函数概览	8
表 3: 参考文档	27
表 4: 术语缩写	27

1 引言

移远通信 EC2x 系列、EG9x 系列、EG2x-G 和 EM05 系列模块支持 QuecOpen®方案；QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档主要介绍移远通信 EC2x 系列、EG9x 系列、EG2x-G 和 EM05 系列 QuecOpen®模块的语音服务相关 API 及语音服务示例。

1.1. 适用模块

表 1: 适用模块

模块系列	模块
EC2x	EC25 系列
	EC21 系列
	EC20-CE
	EC20-CN
EG9x	EG95 系列
	EG91 系列
EG2x-G	EG25-G
	EG21-G
EM05	EM05 系列

2 语音服务相关 API

2.1. 头文件

语音服务接口头文件位于 SDK 中的 `ql-ol-sdk\qo-ol-extsdk\include` 目录下, `ql_mcm_voice.h` 文件即语音服务 API 的头文件。若无特别说明, 本文档所提到的头文件均在该目录下。

2.2. 函数概览

表 2: 函数概览

函数名	说明
<code>QL_Voice_Call_Client_Init()</code>	初始化语音通话功能并获取使用句柄
<code>QL_Voice_Call_Client_Deinit()</code>	销毁语音通话功能资源
<code>QL_Voice_Call_AddStateHandler()</code>	注册上报语音通话状态的回调函数
<code>QL_Voice_Call_RemoveStateHandler()</code>	销毁注册上报电话状态的回调函数
<code>QL_Voice_Call_Start()</code>	主叫端拨打电话
<code>QL_Voice_Call_End()</code>	挂断或者拒接电话
<code>QL_Voice_Call_Answer()</code>	接听电话
<code>QL_Voice_Call_GetWaitingStatus()</code>	设置呼叫等待状态
<code>QL_Voice_Call_SetWaiting()</code>	获取模块呼叫等待状态
<code>QL_Voice_Call_Hold()</code>	将来电设置为呼叫保持状态
<code>QL_Voice_Call_UnHold()</code>	取消呼叫保持
<code>QL_Voice_Call_CancelDial()</code>	结束所有会话

<code>QL_Voice_Call_Dtmf()</code>	播放 DTMF 音
<code>QL_Voice_Call_SetForwarding()</code>	设置呼叫转移模式
<code>QL_Voice_Call_GetForwardingStatus()</code>	获取所有号码的呼叫转移模式状态

2.3. 函数描述

2.3.1. QL_Voice_Call_Client_Init

该函数用于初始化语音通话功能并获取一个使用句柄。

- 函数原型

```
int QL_Voice_Call_Client_Init(voice_client_handle_type *ph_voice);
```

- 参数

ph_voice:

[Out] 语音通话句柄指针。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.1.1. voice_client_handle_type

语音通话句柄定义如下：

```
typedef uint32 voice_client_handle_type;
```

2.3.2. QL_Voice_Call_Client_Deinit

该函数用于销毁语音通话功能资源。

- 函数原型

```
int QL_Voice_Call_Client_Deinit(voice_client_handle_type h_voice);
```

- 参数

h_voice:

[In] 语音通话句柄。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.3. QL_Voice_Call_AddStateHandler

该函数用于注册上报语音通话状态的回调函数。

- 函数原型

```
int QL_Voice_Call_AddStateHandler(voice_client_handle_type h_voice, QL_VoiceCall_StateHandlerFunc_t handlerPtr, void* contextPtr);
```

- 参数

h_voice:

[In] 语音通话句柄。

handlerPtr:

[In] 指向注册语音通话状态回调函数的指针，回调函数详见第2.3.3.1章。

contextPtr:

[In] 获取来电时的通话 ID。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.3.1. QL_VoiceCall_StateHandlerFunc_t

该回调函数用于指示语音通话状态改变。

- 函数原型

```
typedef void (*QL_VoiceCall_StateHandlerFunc_t)
(
    int call_id,
    char* phone_num,
```

```
E_QL_VOICE_CALL_STATE_T state,
void *contextPtr
);
```

● 参数

call_id:

[In] 通话 ID。

phone_num:

[In] 被叫号码。

state:

[In] 语音通话状态。详见第2.3.3.2章枚举 *ql_mcm_voice_call_state_t*。

contextPtr:

[In] 该参数无意义，暂不支持。

● 返回值

无

2.3.3.2. ql_mcm_voice_call_state_t

电话状态枚举定义如下：

```
typedef enum
{
    E_QL_MCM_VOICE_CALL_STATE_INCOMING    = 0x0000,
    E_QL_MCM_VOICE_CALL_STATE_DIALING     = 0x0001,
    E_QL_MCM_VOICE_CALL_STATE_ALERTING    = 0x0002,
    E_QL_MCM_VOICE_CALL_STATE_ACTIVE      = 0x0003,
    E_QL_MCM_VOICE_CALL_STATE_HOLDING     = 0x0004,
    E_QL_MCM_VOICE_CALL_STATE_END         = 0x0005,
    E_QL_MCM_VOICE_CALL_STATE_WAITING
}ql_mcm_voice_call_state_t;
```

● 成员

成员	描述
<i>E_QL_MCM_VOICE_CALL_STATE_INCOMING</i>	有电话呼入
<i>E_QL_MCM_VOICE_CALL_STATE_DIALING</i>	正在拨号

<code>E_QL_MCM_VOICE_CALL_STATE_ALERTING</code>	被叫呼叫等待，主叫振铃中
<code>E_QL_MCM_VOICE_CALL_STATE_ACTIVE</code>	正在通话
<code>E_QL_MCM_VOICE_CALL_STATE_HOLDING</code>	呼叫保持
<code>E_QL_MCM_VOICE_CALL_STATE_END</code>	断开通话连接
<code>E_QL_MCM_VOICE_CALL_STATE_WAITING</code>	呼叫等待

2.3.4. QL_Voice_Call_RemoveStateHandler

该函数用于销毁注册上报电话状态的回调函数。

- 函数原型

```
ql_vc_errcode_e ql_voice_call_end(void);
```

- 参数

`h_voice`:

[In] 语音通话句柄。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.5. QL_Voice_Call_Start

该函数用于主叫端拨打电话。

- 函数原型

```
int QL_Voice_Call_Start(voice_client_handle_type h_voice, E_QL_VCALL_ID_T simId, char* phone_number, int *call_id);
```

- 参数

`h_voice`:

[In] 语音通话句柄。

`simId`:

[In] (U)SIM 卡槽 ID。此参数当前不生效。

phone_number:

[In] 被叫方手机号码。

call_id:

[Out] 本次通话的 ID。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.6. QL_Voice_Call_End

该函数用于挂断或者拒接电话。

- 函数原型

```
int QL_Voice_Call_End(voice_client_handle_type h_voice, int call_id);
```

- 参数

h_voice:

[In] 语音通话句柄。

call_id:

[In] 本次通话的 ID。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.7. QL_Voice_Call_Answer

该函数用于接听电话。

- 函数原型

```
int QL_Voice_Call_Answer(voice_client_handle_type h_voice, int call_id);
```

- 参数

h_voice:

[In] 语音通话句柄。

call_id:

[In] 本次通话的 ID。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.8. QL_Voice_Call_SetWaiting

该函数用于设置呼叫等待状态。

- 函数原型

```
int QL_Voice_Call_SetWaiting(int h_voice, ql_mcm_voice_call_waiting_service_t *pe_service );
```

- 参数

h_voice:

[In] 语音通话句柄。

pe_service:

[In] 呼叫等待状态。详见第2.3.8.1章枚举 *E_QL_MCM_VOICE_CALL_WAITING_SERVICE_T*。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.8.1. E_QL_MCM_VOICE_CALL_WAITING_SERVICE_T

呼叫等待状态枚举定义如下：

```
typedef enum
{
    E_QL_MCM_VOICE_CALL_WAITING_VOICE_ENABLED    = 0,
    E_QL_MCM_VOICE_CALL_WAITING_DISABLED         = 1,
}E_QL_MCM_VOICE_CALL_WAITING_SERVICE_T;
```

- 成员

成员	描述
<i>E_QL_MCM_VOICE_CALL_WAITING_VOICE_ENABLED</i>	呼叫等待已启用
<i>E_QL_MCM_VOICE_CALL_WAITING_DISABLED</i>	呼叫等待已禁用

2.3.9. QL_Voice_Call_GetWaitingStatus

该函数用于获取模块呼叫等待状态。

- 函数原型

```
int QL_Voice_Call_GetWaitingStatus(int h_voice, ql_mcm_voice_call_waiting_service_t
*pe_service );
```

- 参数

h_voice:

[In] 语音通话句柄。

pe_service:

[Out] 当前模块的语音呼叫等待状态。详见第2.3.8.1章枚举 *E_QL_MCM_VOICE_CALL_WAITING_SERVICE_T*。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.10. QL_Voice_Call_Hold

该函数用于将来电设置为呼叫保持状态。

- 函数原型

```
int QL_Voice_Call_Hold(voice_client_handle_type h_voice);
```

- 参数

h_voice:

[In] 语音通话句柄。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.11. QL_Voice_Call_UnHold

该函数用于取消呼叫保持。

- 函数原型

```
int QL_Voice_Call_UnHold(voice_client_handle_type h_voice);
```

- 参数

h_voice:

[In] 语音通话句柄。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.12. QL_Voice_Call_CancelDial

该函数用于结束所有会话。

- 函数原型

```
int QL_Voice_Call_CancelDial(voice_client_handle_type h_voice);
```

- 参数

h_voice:

[In] 语音通话句柄。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.13. QL_Voice_Call_Dtmf

该函数用于播放 DTMF 音。

- 函数原型

```
int QL_Voice_Call_Dtmf(voice_client_handle_type h_voice, uint8_t digit, int call_id);
```

● 参数

h_voice:

[In] 语音通话句柄。

digit:

[In] DTMF 音对应的数字类型的 ASCII 码。

call_id:

[In] 本次通话的 ID。

● 返回值

0 函数执行成功

其他值 函数执行失败

2.3.14. QL_Voice_Call_SetForwarding

该函数用于设置呼叫转移模式。

● 函数原型

```
int QL_Voice_Call_SetForwarding(int h_voice, E_QL_MCM_VOICE_CALL_SERVICE_T service,
E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T reason, char *number);
```

● 参数

h_voice:

[In] 语音通话句柄。

service:

[In] 呼叫转移配置。详见第2.3.14.1章枚举 E_QL_MCM_VOICE_CALL_SERVICE_T。

reason:

[In] 呼叫转移类型。详见第2.3.14.2章 E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T。

number:

[In] 被叫号码。

● 返回值

0 函数执行成功

其他值 函数执行失败

2.3.14.1. E_QL_MCM_VOICE_CALL_SERVICE_T

呼叫转移配置枚举类型定义如下：

```
typedef enum
{
    E_QL_MCM_VOICE_SERVICE_UNKOWN        = 0,
    E_QL_MCM_VOICE_SERVICE_REGISTER      = 1,
    E_QL_MCM_VOICE_SERVICE_ERASE         = 2,
}E_QL_MCM_VOICE_CALL_SERVICE_T;
```

● 成员

成员	描述
<i>E_QL_MCM_VOICE_SERVICE_UNKOWN</i>	未知操作
<i>E_QL_MCM_VOICE_SERVICE_REGISTER</i>	注册呼叫转移
<i>E_QL_MCM_VOICE_SERVICE_ERASE</i>	删除呼叫转移号码

2.3.14.2.E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T

呼叫转移类型枚举定义如下：

```
typedef enum
{
    E_QL_MCM_VOICE_CALL_FORWARD_UNCONDITIONALLY    = 0,
    E_QL_MCM_VOICE_CALL_FORWARD_MOBILEBUSY         = 1,
    E_QL_MCM_VOICE_CALL_FORWARD_NOREPLY            = 2,
    E_QL_MCM_VOICE_CALL_FORWARD_UNREACHABLE        = 3,
    E_QL_MCM_VOICE_CALL_FORWARD_ALLFORWARDING      = 4,
    E_QL_MCM_VOICE_CALL_FORWARD_ALLCONDITIONAL     = 5,
}E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T;
```

● 成员

成员	描述
<i>E_QL_MCM_VOICE_CALL_FORWARD_UNCONDITIONALLY</i>	无条件呼叫转移
<i>E_QL_MCM_VOICE_CALL_FORWARD_MOBILEBUSY</i>	被叫忙
<i>E_QL_MCM_VOICE_CALL_FORWARD_NOREPLY</i>	无响应

<code>E_QL_MCM_VOICE_CALL_FORWARD_UNREACHABLE</code>	不可及呼叫转移
<code>E_QL_MCM_VOICE_CALL_FORWARD_ALLFORWARDING</code>	所有呼叫转移（0~3）
<code>E_QL_MCM_VOICE_CALL_FORWARD_ALLCONDITIONAL</code>	所有条件呼叫转移（1~3）

2.3.15. QL_Voice_Call_GetForwardingStatus

该函数用于获取呼叫转移模式状态。

- 函数原型

```
int QL_Voice_Call_GetForwardingStatus(int h_voice, E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T reason, ql_mcm_voice_call_forwarding_status_list_t *pt_status);
```

- 参数

h_voice:

[In] 语音通话句柄。

reason:

[In] 呼叫转移类型。详见第2.3.14.2章 `E_QL_MCM_VOICE_CALL_FORWARDING_REASON_T`。

pt_status:

[Out] 呼叫转移状态列表。详见第2.3.15.1章 `ql_mcm_voice_call_forwarding_status_list_t`。

- 返回值

0 函数执行成功

其他值 函数执行失败

2.3.15.1. ql_mcm_voice_call_forwarding_status_list_t

呼叫转移状态列表结构体定义如下：

```
typedef struct
{
    QL_MCM_VOICE_CALL_FORWARDING_STATUS_T    status;
    uint32_t                                  info_len;
    QL_MCM_VOICE_CALL_FORWARDING_INFO_T
    info[QL_MCM_MAX_CALL_FORWARDING_INFO];
}ql_mcm_voice_call_forwarding_status_list_t;
```

● 参数

类型	参数	描述
<code>QL_MCM_VOICE_CALL_FORWARDING_STATUS_T</code>	<code>status</code>	呼叫转移状态。枚举详见第2.3.15.2章。
<code>uint32_t</code>	<code>info_len</code>	<code>info</code> 数组的有效个数。
<code>QL_MCM_VOICE_CALL_FORWARDING_INFO_T</code>	<code>info</code>	呼叫转移信息。枚举详见第2.3.15.3章。

2.3.15.2.QL_MCM_VOICE_CALL_FORWARDING_STATUS_T

呼叫转移状态枚举定义如下：

```
typedef enum
{
    E_QL_MCM_VOICE_CALL_FORWARDING_DISABLED    = 0,
    E_QL_MCM_VOICE_CALL_FORWARDING_ENABLED    = 1,
}QL_MCM_VOICE_CALL_FORWARDING_STATUS_T;
```

● 成员

成员	描述
<code>E_QL_MCM_VOICE_CALL_FORWARDING_DISABLED</code>	呼叫转移已禁用
<code>E_QL_MCM_VOICE_CALL_FORWARDING_ENABLED</code>	呼叫转移已启用

2.3.15.3.QL_MCM_VOICE_CALL_FORWARDING_INFO_T

呼叫转移信息结构体定义如下：

```
typedef struct
{
    QL_MCM_VOICE_CALL_FORWARDING_TYPE_T    type;
    char number[QL_MCM_MAX_PHONE_NUMBER + 1];
}QL_MCM_VOICE_CALL_FORWARDING_INFO_T;
```

● 成员

类型	参数	描述
<code>QL_MCM_VOICE_CALL_FORWARDING_TYPE_T</code>	<code>type</code>	呼叫转移类型。详见第2.3.15.4章
<code>char</code>	<code>number</code>	呼叫转移号码

2.3.15.4.QL_MCM_VOICE_CALL_FORWARDING_TYPE_T

呼叫转移类型枚举定义如下：

```
typedef enum
{
    E_QL_MCM_VOICE_CALL_FORWARDING_TYPE_VOICE      = 0,
    E_QL_MCM_VOICE_CALL_FORWARDING_TYPE_DATA       = 1,
    E_QL_MCM_VOICE_CALL_FORWARDING_TYPE_VOICE_DATA = 2,
}QL_MCM_VOICE_CALL_FORWARDING_TYPE_T;
```

● 成员

成员	描述
<code>E_QL_MCM_VOICE_CALL_FORWARDING_TYPE_VOICE</code>	语音类型
<code>E_QL_MCM_VOICE_CALL_FORWARDING_TYPE_DATA</code>	数据类型
<code>E_QL_MCM_VOICE_CALL_FORWARDING_TYPE_VOICE_DATA</code>	语音和数据类型

3 语音通话使用步骤示例

语音通话的详细使用步骤请参考 `example/voice/example_voice.c`。

- 步骤1:** 首先，调用 `QL_Voice_Call_Client_Init()`完成初始化。必须首先执行初始化步骤方可进行下一步；
- 步骤2:** 调用 `QL_Voice_Call_AddStateHandler()`注册上报语音通话状态的回调函数；
- 步骤3:** 通信双方语音通话，完成通话（可以由任何一方挂断）；
- 步骤4:** 调用 `QL_Voice_Call_RemoveStateHandler()`销毁注册上报电话状态的回调函数；
- 步骤5:** 调用 `QL_Voice_Call_Client_Deinit()`销毁语音通话功能资源。

上述步骤中，**步骤 4** 和**步骤 5** 在退出程序前执行。

4 编译说明

编译单个 `example_voice.c` 步骤如下：

步骤1： 执行命令 `tar -jxvf ql-ol-sdk.tar.bz2` 解压 `ql-ol-sdk.tar.bz2`；

步骤2： 执行命令 `cd ql-ol-sdk` 进入 `ql-ol-sdk` 目录；

步骤3： 执行命令 `source ql-ol-crosstool/ql-ol-crosstool-env-init` 初始化编译环境。此步骤是为了确保 SDK 版本与模块固件版本一致，否则可能出现错误。

步骤4： 执行命令 `cd ql-ol-extsdk/example/voice`；

步骤5： 执行 `make clean;make` 清除编译结果。

5 语音通话演示步骤

执行如下命令，可以开始进行语音通话演示。

```
root@mdm9607-perf:~# ./example_voice
```

5.1. 拨打电话

```
please input cmd index(-1 exit): 1
please input dest phone number:
12100000000
QL_Voice_Call_Start ret = 0, with voice_call_id=1
//下面此类 log 为电话状态变化
please input cmd index(-1 exit): ##### Call id=1, PhoneNum:12100000000, event=DIALING!
##### //发起
##### Call id=1, PhoneNum:12100000000, event=ALERTING! ##### //对方振铃
##### Call id=1, PhoneNum:12100000000, event=ACTIVE! #####//电话建立连接
##### Call id=1, PhoneNum:12100000000, event=END! #####//电话结束
0
```

5.2. 接听并挂断电话

```
please input cmd index(-1 exit): ##### Call id=1, PhoneNum:12100000000, event=INCOMING!
#####
3
please input answer call id
1
##### Call id=1, PhoneNum:12100000000, event=ACTIVE! #####
ret = 0

please input cmd index(-1 exit): ##### Call id=1, PhoneNum:12100000000, event=END!
#####
0
Supported test cases:
0: print_help
```

```

1: QL_Voice_Call_Start
2: QL_Voice_Call_End
3: QL_Voice_Call_Answer
4: QL_Voice_Call_CancelDial
5: QL_Voice_Call_Dtmf
6: QL_Voice_Call_SetForwarding
7: QL_Voice_Call_GetForwardingStatus
8: QL_Voice_Call_SetWaiting
9: QL_Voice_Call_GetWaitingStatus
10: QL_Voice_Call_Hold
11: QL_Voice_Call_UnHold

please input cmd index(-1 exit): ##### Call id=1, PhoneNum:12100000000, event=INCOMING!
#####
3
  please input answer call id
1
##### Call id=1, PhoneNum:12100000000, event=ACTIVE! #####
  ret = 0

//挂断电话
please input cmd index(-1 exit): 2
please input end call id:
1
##### Call id=1, PhoneNum:12100000000, event=END! #####
  ret = 0

```

5.3. 保持呼叫

主叫 A（开通了呼叫等待业务）拨打 B（12100000000），并且 B 接听了电话，此时调用 `QL_Voice_Call_Hold` 可以让 B 处于呼叫保持状态，再次调用 `QL_Voice_Call_Hold` 可以恢复与 B 的通话。

```

please input cmd index(-1 exit): 0
Supported test cases:
0: print_help
1: QL_Voice_Call_Start
2: QL_Voice_Call_End
3: QL_Voice_Call_Answer
4: QL_Voice_Call_CancelDial
5: QL_Voice_Call_Dtmf

```

```
6: QL_Voice_Call_SetForwarding
7: QL_Voice_Call_GetForwardingStatus
8: QL_Voice_Call_SetWaiting
9: QL_Voice_Call_GetWaitingStatus
10: QL_Voice_Call_Hold
11: QL_Voice_Call_UnHold

please input cmd index(-1 exit): ##### Call id=1, PhoneNum:12100000000, event=INCOMING!
#####
3
  please input answer call id
1
##### Call id=1, PhoneNum:12100000000, event=ACTIVE! #####
  ret = 0

//呼叫保持，挂起当前通话
please input cmd index(-1 exit): 10
##### Call id=1, PhoneNum:12100000000, event=HOLDING! #####
  QL_Voice_Call_Hold ret = 0

please input cmd index(-1 exit): 11
##### Call id=1, PhoneNum:12100000000, event=ACTIVE! #####
  QL_Voice_Call_UnHold ret = 0

please input cmd index(-1 exit): ##### Call id=1, PhoneNum:12100000000, event=END!
#####
```

6 附录 参考文档及术语缩写

表 3: 参考文档

文档名称

[1] EC2x&EG9x&EG2x-G&EM05 系列_QuecOpen_快速开发指导

表 4: 术语缩写

缩写	英文全称	中文全称
API	Application Programming Interface	应用程序编程接口
ASCII	American Standard Code for Information Interchange	美国信息交换标准码
DTMF	Dual-Tone Multifrequency	双音多频
ID	Mostly refers to Identifier in terms of software	软件中多数指“标识符”
IoT	Internet of Things	物联网
SDK	Software Development Kit	软件开发工具包